🧠 1. What is an Inner Class?

An **Inner Class** (also called **Nested Class**) is a class **defined inside another class**.

Think of it like this:

A class (outer) has a smaller class (inner) logically grouped within it.

This helps **organize code** that is **closely related** — making it cleaner and more modular.

🧱 2. Syntax of Inner Class

```
class OuterClass:
    class InnerClass:
        pass
```

🧠 The inner class exists **only in the scope** of the outer class.

💡 3. Example 1: Basic Inner Class

```
class Outer:
    def __init__(self):
        self.outer_name = "Outer Class"

    def show_outer(self):
        print("This is the Outer class method")

    # Inner class defined inside
    class Inner:
        def __init__(self):
            self.inner_name = "Inner Class"

        def show_inner(self):
```

```
        print("This is the Inner class method")
```

Using Inner Class

```
# Access inner class through outer
outer_obj = Outer()
inner_obj = Outer.Inner()

outer_obj.show_outer()
inner_obj.show_inner()
```

📜 Output:

```
This is the Outer class method
This is the Inner class method
```

🚙 4. Example 2: Real-Life Analogy — Car and Engine

Here, **Engine** is logically part of **Car**, not something used alone.
So, defining it inside the Car class makes perfect sense.

```
class Car:
    def __init__(self, brand, model):
        self.brand = brand
        self.model = model
        self.engine = self.Engine()  # Create inner class object

    def show(self):
        print(f"Car: {self.brand}, Model: {self.model}")
        self.engine.show_engine()  # calling inner class method

    class Engine:
```

```
    def __init__(self):
        self.horsepower = 200

    def show_engine(self):
        print(f"Engine Power: {self.horsepower} HP")
```

Using the Inner Class:

```
car1 = Car("Tesla", "Model S")
car1.show()
```

📄 Output:

```
Car: Tesla, Model: Model S
Engine Power: 200 HP
```

✅ The Engine class is **encapsulated** inside Car since it logically belongs there.

🧩 5. Accessing Inner Class from Outside

You can still access it directly if needed:

```
engine_obj = Car.Engine()  # create inner class object directly
engine_obj.show_engine()
```

Output:

```
Engine Power: 200 HP
```

But typically, you create the inner class object **through the outer class**, not separately.

🧠 6. Why Use Inner Classes?

Reason	Explanation
Logical Grouping	Keeps related classes together (e.g., Car → Engine)
Encapsulation	Hides internal details (inner class not visible globally)

Reason	Explanation
Readability	Makes the outer class easier to manage
Namespace Management	Prevents name clashes with other classes

---

💼 7. Example 3: Student and Address

```
class Student:
    def __init__(self, name, rollno):
        self.name = name
        self.rollno = rollno
        self.address = self.Address("Bangalore", "Karnataka")

    def show(self):
        print(f"Student Name: {self.name}, Roll No: {self.rollno}")
        self.address.show_address()

    class Address:
        def __init__(self, city, state):
            self.city = city
            self.state = state

        def show_address(self):
            print(f"Address: {self.city}, {self.state}")
```

Using it:

```
s1 = Student("Alice", 101)
s1.show()
```

📜 Output:

Student Name: Alice, Roll No: 101

Address: Bangalore, Karnataka

✅ *Inner class Address exists only as part of the outer class Student.*

🧪 8. Practice Exercise

Try this yourself 👇
(Write in your editor — no answers, just instructions)

Exercise: Python Inner Class

1. Create a class 'Laptop' with:

- attributes: brand, model

- inner class 'Processor' with attributes: cores, speed

2. In Laptop's constructor, create a Processor object.

3. Add a method in Laptop to display full details (including processor info).

4. Create an object and print laptop details.

📜 9. Summary Table

Concept	Description	Example
Inner Class	Class defined inside another class	class Outer: class Inner:
Access	Outer.Inner()	engine = Car.Engine()
Used For	Logical grouping & encapsulation	Car → Engine, Student → Address
Created From	Either directly or via outer class	self.inner = self.Inner()

🧠 Instructor Tip

Think of **inner classes** like:

🧍 Outer Class → The "main object"

🧩 Inner Class → A "component" or "part" of that main object

For example:

- Car → Engine
- Computer → CPU
- Student → Address
- Company → Department